

# pfSense Integration

BELASRI Ayman

## What is pfSense ?

**pfSense** is a free, open-source firewall and routing software distribution based on the FreeBSD operating system. Designed to be deployed on physical hardware or inside a virtual machine, it transforms standard systems into enterprise-grade network security appliances.

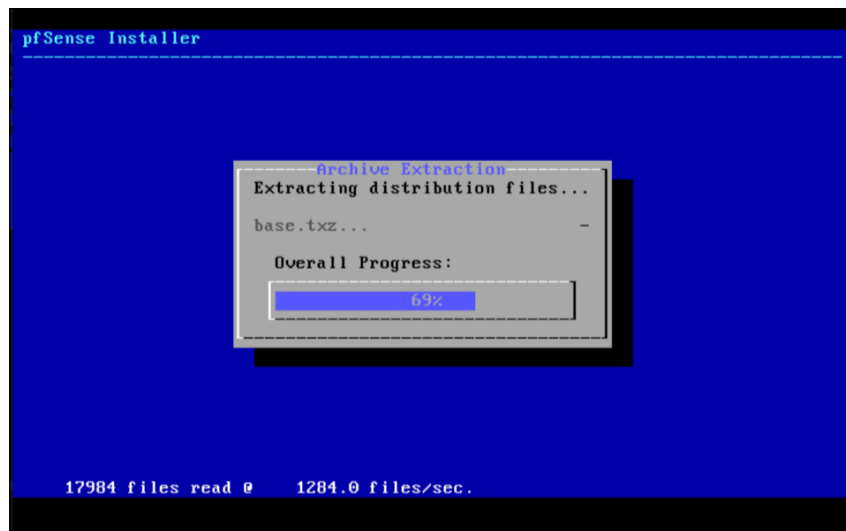
## Why combine it with Wazuh ?

We chose **pfSense** because it's a solid open-source edge firewall that integrates easily with Wazuh. It filters all incoming internet traffic at the perimeter and sends its firewall logs directly to our Wazuh server via syslog.

## pfSense Deployment & Installation

To set up pfSense in our virtual lab environment, we deploy it on a lightweight dedicated Virtual Machine with the following specifications:

- **CPU:** 1 CPU core
  - **RAM:** 1 GB
  - **Storage:** 10 GB
  - **Network Interfaces:**
    - **Adapter 1 (WAN):** NAT (simulates internet access)
    - **Adapter 2 (LAN):** Host-only (isolated internal lab network)
1. First, we grab the pfSense Community Edition ISO file and attach it as the boot installation media in our QEMU VM.
  2. We boot up the QEMU VM , and let the installer partition the disk using standard Auto ZFS.



3. After booting pfSense, we assigned the network interfaces to map directly to our QEMU dual-NIC setup:
  - **WAN (vtnet0):DHCP** : Connects to the host's NAT network to provide internet access for pfSense updates and threat feeds.
  - **LAN (vtnet1):Static IP**: Serves as the default gateway for internal VMs, forcing all lab traffic through pfSense for filtering and logging.

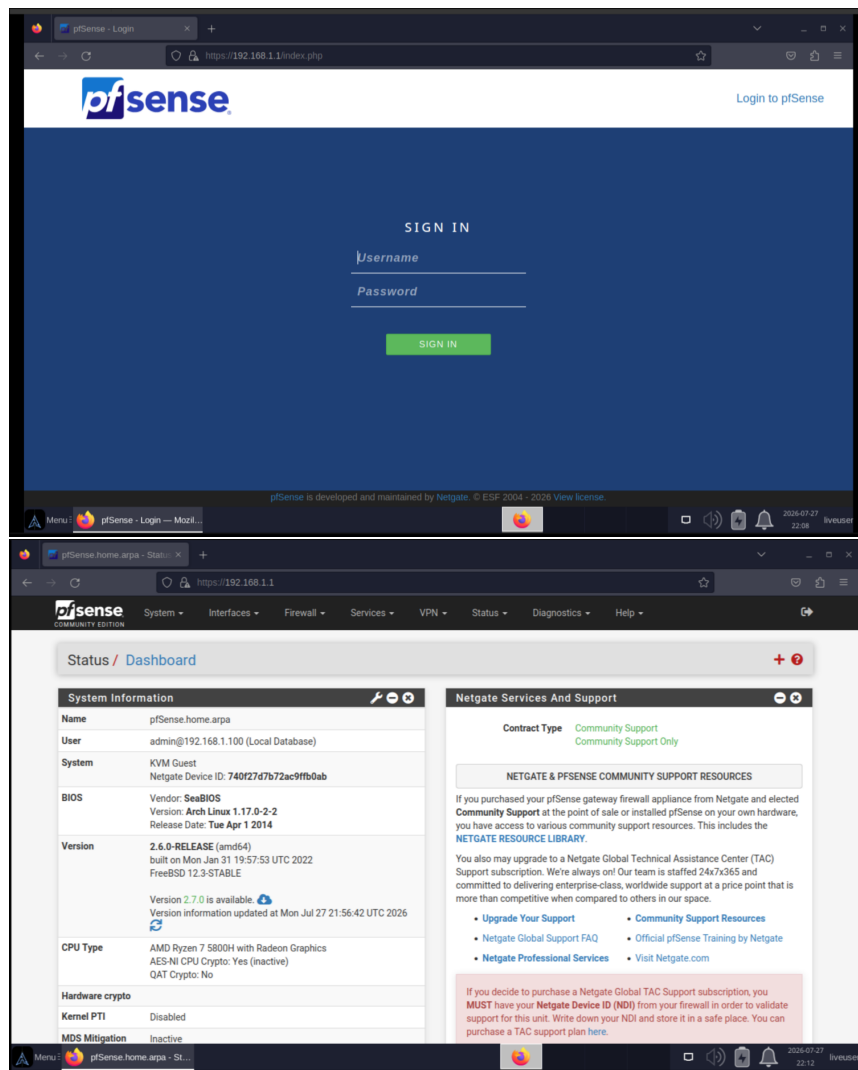
```
FreeBSD/amd64 (pfSense.home.arpa) (ttyv0)
KVM Guest - Netgate Device ID: 1550c3649b17ac609c11
*** Welcome to pfSense 2.6.0-RELEASE (amd64) on pfSense ***

WAN (wan)      -> vtnet0      -> v4/DHCP4: 192.168.122.60/24
LAN (lan)      -> vtnet1      -> v4: 192.168.1.1/24

0) Logout (SSH only)          9) pfTop
1) Assign Interfaces          10) Filter Logs
2) Set interface(s) IP address 11) Restart webConfigurator
3) Reset webConfigurator password 12) PHP shell + pfSense tools
4) Reset to factory defaults    13) Update from console
5) Reboot system              14) Enable Secure Shell (sshd)
6) Halt system                15) Restore recent configuration
7) Ping host                  16) Restart PHP-FPM
8) Shell

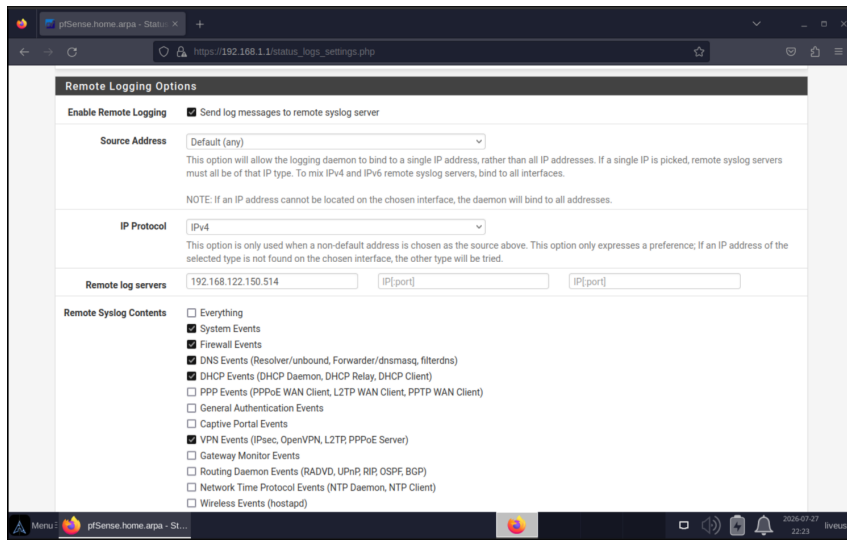
Enter an option: █
```

4. We open a browser on another VM on the LAN, navigate to the LAN IP, log in using (admin:pfsense), and complete the setup wizard.



## pfSense Configuration

1. First we need to enable remote logging so that pfSense can forward syslog data directly to our Wazuh server IP using the default syslog port 514.



2. Then we edit the Wazuh Manager's `/var/ossec/etc/ossec.conf` file to accept incoming UDP syslog traffic on port 514.

```
<ossec_config>
  <!-- pfSense syslog input -->
  <remote>
    <connection>syslog</connection>
    <port>514</port>
    <protocol>udp</protocol>
    <allowed-ips>PFSENSE-IP</allowed-ips>
    <local_ip>WAZUH-SERVER-IP</local_ip>
  </remote>
</ossec_config>
```

3. Next, we create a custom decoder in `/var/ossec/etc/decoders/local_decoder.xml` to extract structured fields like source IP, destination IP, and protocol from raw pfSense logs. filterlog

```
<decoder name="pfsense-custom">
  <prematch>filterlog</prematch>
</decoder>

<decoder name="pfsense-fields">
  <parent>pfsense-custom</parent>
  <regex>(\w+)\d+:\s*,\s*,\s*,(\S+),\s*,\s*,(\S+),\s*,\s*,\s*,\s*,\s*,\s*,\s*,\s*,(\S+),\s*,(\S+),(\S+);(\d+),\s*<regex>
  <order>logsource,id,section,protocol,srcip,dstip,srcport,dstport</order>
</decoder>
```

4. Lastly, we will add custom rules in `/var/ossec/etc/rules/local_rules.xml`

to generate alerts and assign severity levels to specific events detected in the decoded pfSense logs :

- **Level 3:** Successful login event.
- **Level 5:** Allowed network traffic.
- **Level 7:** Blocked firewall connection.
- **Level 10:** Failed login attempt.

```
<group name="pfsense,firewall,">

  <!-- Parent / Generic pfSense Log -->
  <rule id="100010" level="3">
    <decoded_as>pfsense-custom</decoded_as>
    <description>pfSense log received</description>
  </rule>

  <!-- Successful Login -->
  <rule id="100011" level="3">
    <if_sid>100010</if_sid>
    <match>Successful login</match>
    <description>pfSense : Successful login from $(srcip)</description>
  </rule>

  <!-- Allowed Traffic -->
  <rule id="100012" level="5">
    <if_sid>100010</if_sid>
    <match>pass</match>
    <description>pfSense : Allowed traffic from $(srcip) to $(dstip)</description>
  </rule>

  <!-- Blocked Traffic -->
  <rule id="100013" level="7">
    <if_sid>100010</if_sid>
    <match>block</match>
    <description>pfSense : Blocked traffic from $(srcip) to $(dstip)</description>
  </rule>

  <!-- Authentication Error -->
  <rule id="100014" level="10">
    <if_sid>100010</if_sid>
    <match>authentication error</match>
    <description>pfSense:Authentication error for user $(dstuser) from $(srcip)</description>
  </rule>

</group>
```

## Conclusion

By combining pfSense with Wazuh, we achieved **centralized visibility, automated log parsing, and real-time, severity-based security alerting** across all firewall and access events.